Entropy in Conway's Game of Life

Measuring Information, Order, and Complexity in 2D Cellular Automata

From Conway to LangGraph • Week 2

Prof. Mirko Degli Esposti • Dept. of Physics • University of Bologna

Why Entropy for the Game of Life?

The Question

A random 'soup' on a 100×100 grid starts with ~50% alive cells. After 1000 generations it settles into a sparse landscape of still lifes, oscillators, and occasional gliders.

How do we quantify this transition from disorder to structure?

What Entropy Reveals

Self-organization: entropy decreases as patterns emerge from random initial conditions

Phase classification: distinguish ordered, chaotic, and complex regimes

Approach to equilibrium: analogies with statistical mechanics and the 2nd law

Complexity signatures: why GoL sits at a special point in rule space

Recap: Shannon Entropy

from 1D to the general framework

$$H(X) = -\Sigma_i \; p(x_i) \log_2 p(x_i) \quad [\text{bits}]$$

H = 0

Perfect order
All cells identical
Fully predictable

H = 1 bit

Maximum uncertainty
for binary variable
(fair coin / 50% density)

0 < H < 1

Partial information
Structure present
Patterns detectable

Part I: Entropy Measures

Adapting information theory from 1D chains to 2D lattices

Density Entropy

The simplest measure: global alive/dead ratio

$$\rho(t) = N_alive(t) / N_total$$

$$H_density(t) = -\rho \log_2 \rho - (1-\rho) \log_2(1-\rho)$$

→ This is the binary entropy function h(ρ)

Key observation

Random soup starts at ρ ≈ 0.5
(H = 1 bit, maximum)

Settles to ρ ≈ 0.0287
(Flammenkamp's constant)
→ H ≈ 0.20 bits

Limitation

Density entropy is coarse-grained — it ignores spatial structure entirely. A random configuration with ρ = 0.03 has the same density entropy as a field of block still lifes at ρ = 0.03, yet they are vastly different in terms of information content and spatial organization.
We need measures that capture spatial correlations.

2D Block Entropy

Extending block entropy to square patches

Definition

Extract all $k \times k$ sub-blocks from the grid.
Each block is a binary pattern $b \in \{0,1\}^{(k^2)}$

$$H_k = -\sum_b p(b) \log_2 p(b)$$

Entropy per cell: $s_k = H_k / k^2$

Maximum: $H_k = k^2$ bits (all $2^{(k^2)}$ patterns equally likely)

Typical block sizes

2×2 → $2^4 = 16$ patterns

3×3 → $2^9 = 512$ patterns

4×4 → $2^{16} = 65536$ patterns (needs larger grids for good statistics)

Physics analogy: 2D block entropy is the discrete analogue of the configurational entropy in lattice statistical mechanics. As $k \to \infty$, s_k converges to the topological entropy of the CA — a measure of the growth rate of the number of admissible configurations.

Neighborhood Pattern Entropy

The natural measure for GoL: Moore neighborhood configurations

Idea

For each cell, look at its Moore neighborhood:
the cell itself + 8 neighbors = 9-bit configuration.

There are $2^9 = 512$ possible neighborhood states.
Of these, GoL's rule only uses the alive-count sum (totalistic), so there are 10 effective classes (0–9 alive).

We can compute entropy at two levels:

Full pattern entropy

All 512 configurations
Max = 9 bits

Sum-based entropy

10 sum classes (0–9)
Max = $\log_2(10) \approx 3.32$ bits

Moore Neighborhood

center cell

+ 8 neighbors = 9 bits

Temporal Entropy

Tracking how each cell fluctuates over time

For a single cell (i,j) over T time steps:

$$f_{ij} = (1/T)\,\Sigma_t\; c_{ij}(t) \quad \text{(fraction of time alive)}$$

$$H_{ij} = -f_{ij}\log_2 f_{ij} - (1-f_{ij})\log_2(1-f_{ij})$$

Average temporal entropy:

$$\langle H_temp\rangle = (1/N)\,\Sigma_{ij}\; H_{ij}$$

Interpretation

$H_{ij} = 0$: cell is always dead or always alive (still life / dead zone)

$H_{ij} = 1$: cell flips with equal probability (edge of chaos)

$H_{ij} \approx h(1/p)$: cell is part of a period-p oscillator

Spatial entropy map: Plotting H_{ij} as a heatmap reveals the 'activity regions' of a GoL configuration. Still lifes appear as cold (H=0) islands; oscillators create warm spots; glider tracks show as faint streaks. This is a powerful visualization tool for your notebooks!

Entropy Dynamics in GoL

What happens when you start from a random soup?

Phase I

t = 0–10

Chaos

Random initial config.
H ≈ maximum.
Rapid transient dynamics.

→

Phase II

t = 10–100

Self-Organization

Entropy drops sharply.
Structures crystallize.
Local patterns emerge.

→

Phase III

t = 100–500

Methuselah Decay

Long-lived transients slowly resolve. Gliders carry away entropy.

→

Phase IV

t > 500

Equilibrium

Stable configuration.
H approaches minimum.
$\rho \to 0.0287$
(Flammenkamp)

Anti-Entropic Behavior

Why GoL is special: entropy decreases without external intervention

The Apparent Paradox

In thermodynamics, the 2nd law dictates:
$\Delta S \geq 0$ (entropy never decreases)

Yet in GoL, starting from maximum entropy (random soup), we observe:

H(t) monotonically decreasing
Spatial order spontaneously emerging
Complex structures crystallizing

How is this possible?

The Resolution

GoL is not a Hamiltonian system:

1. Non-reversible: many-to-one mapping (multiple predecessors → one successor)

2. Dissipative: information is destroyed at every time step (no conservation law)

3. No equilibrium dynamics: GoL is an open system in the information-theoretic sense — it 'radiates' entropy

→ The 2nd law applies to closed systems. GoL is effectively dissipative.

GoL as an Outlier in Rule Space

Entropy signatures distinguish 'lifelike' rules (B3/S23) from the 2^{18} alternatives

Criticality

GoL operates near a critical point between ordered (Class I/II) and chaotic (Class III) regimes. Langton's $\lambda \approx 0.273$ for B3/S23 places it in the 'edge of chaos' region.

Anti-Entropy

Starting from random soups, GoL exhibits a stronger entropy decrease than almost all other life-like rules. Most rules either die out completely or remain chaotic.

Integration

GoL configurations show high integrated information (Φ): local structures are correlated across the grid. This is a hallmark of complex, 'lifelike' systems.

Ref: arxiv:2410.22389 — 'Conway's Game of Life as an Analogue to a Habitable World' (2024)

Part II: Implementation

NumPy-vectorized entropy computations for your notebooks

GoL Engine (Recap)

Convolution-based update — fully vectorized

```
import numpy as np
from scipy.signal import convolve2d

KERNEL = np.array([[1,1,1],
                   [1,0,1],
                   [1,1,1]])

def step(grid):
    """One GoL generation (periodic BC)."""
    neighbors = convolve2d(grid, KERNEL,
                           mode='same', boundary='wrap')
    return ((neighbors == 3) |
            (grid & (neighbors == 2))).astype(int)

def evolve(grid, steps):
    """Return full spacetime history."""
    history = np.zeros((steps+1, *grid.shape), dtype=int)
    history[0] = grid
    for t in range(steps):
        history[t+1] = step(history[t])
    return history
```

Notes

Key design choices:

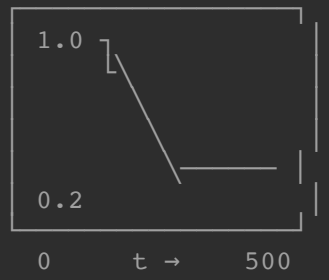
- scipy.convolve2d handles the neighbor counting
- Periodic boundary cond. (torus topology)
- Boolean logic avoids branching → fast on large grids
- History array enables temporal analysis

Implementing Density Entropy

```
def density_entropy(grid):
    """Binary entropy of alive fraction."""
    rho = grid.mean()
    if rho == 0 or rho == 1:
        return 0.0
    return -rho * np.log2(rho) - (1-rho) * np.log2(1-rho)

def entropy_trajectory(history):
    """H_density(t) for full spacetime history."""
    return np.array([density_entropy(h) for h in history])

# Example: random 100x100 soup, 500 steps
grid = (np.random.random((100, 100)) < 0.5).astype(int)
history = evolve(grid, 500)
H = entropy_trajectory(history)

# Plot
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 4))
plt.plot(H, color='steelblue', linewidth=1.5)
plt.xlabel('Generation'); plt.ylabel('H_density [bits]')
plt.title('Density Entropy: Random Soup → Equilibrium')
plt.axhline(y=H[-1], color='gold', ls='--',
            label=f'H_eq ≈ {H[-1]:.3f}')
plt.legend(); plt.grid(alpha=0.3)
```

Expected Output

Characteristic curve:



Sharp drop in first ~50 generations, then slow convergence to H_eq ≈ 0.20 bits

Implementing 2D Block Entropy

```python
def block_entropy_2d(grid, k=2):
    """Shannon entropy of k×k block patterns.
    Vectorized extraction via stride tricks."""
    # Extract all k×k blocks (no loops!)
    blocks = np.lib.stride_tricks.sliding_window_view(
        grid, (k, k)
    )  # shape: (rows-k+1, cols-k+1, k, k)

    # Flatten and convert to integer keys
    flat = blocks.reshape(-1, k*k)
    keys = flat @ (2 ** np.arange(k*k))

    # Count frequencies → probabilities → entropy
    counts = np.bincount(keys, minlength=2**(k*k))
    probs = counts / counts.sum()
    probs = probs[probs > 0]
    return -np.sum(probs * np.log2(probs))

# Usage
for k in [2, 3, 4]:
    H_r = block_entropy_2d(random_grid, k)
    H_s = block_entropy_2d(settled_grid, k)
    print(f"k={k}: random={H_r:.1f}, settled={H_s:.1f}")
```

Key Insight

The stride_tricks approach avoids explicit loops — crucial for large grids.

Binary encoding (base-2 → integer) makes hashing O(1) via bincount.

Expected output:
k=2: ~4.0 → ~2.1
k=3: ~9.0 → ~3.5
k=4: ~16 → ~5.2

Larger blocks capture more spatial structure.

Implementing Neighborhood Entropy

```
def neighborhood_entropy(grid, mode='sum'):
    """
    Entropy of Moore neighborhood configurations.
    mode='sum'  : 10 classes (neighbor count 0-9)
    mode='full' : 512 classes (all 9-bit patterns)
    """
    kernel_full = np.ones((3, 3), dtype=int)

    if mode == 'sum':
        # Count alive neighbors + self
        total = convolve2d(grid, kernel_full,
                           mode='same', boundary='wrap')
        counts = np.bincount(total.ravel(), minlength=10)
    else:
        # Encode full 3x3 neighborhood as 9-bit integer
        patches = np.lib.stride_tricks.sliding_window_view(
            np.pad(grid, 1, mode='wrap'), (3, 3)
        )
        powers = 2 ** np.arange(9).reshape(3, 3)
        keys = (patches * powers).sum(axis=(-2, -1))
        counts = np.bincount(keys.ravel(), minlength=512)
```

Note: The 'sum' mode captures GoL-relevant structure (since the rule is totalistic). The 'full' mode is more general and captures spatial anisotropy in the patterns.

```
    return -np.sum(probs * np.log2(probs))
```

Implementing Temporal Entropy

```
def temporal_entropy_map(history):
    """
    Compute per-cell temporal entropy over a history.
    Returns 2D array of H_ij values.
    """
    T = len(history)
    # Fraction of time each cell is alive
    f = history.mean(axis=0)   # shape: (rows, cols)

    # Binary entropy, handling 0 and 1 edges
    H = np.zeros_like(f)
    mask = (f > 0) & (f < 1)
    H[mask] = (-f[mask] * np.log2(f[mask])
               - (1-f[mask]) * np.log2(1-f[mask]))
    return H

# Example: visualize activity map
grid = (np.random.random((100, 100)) < 0.5).astype(int)
history = evolve(grid, 200)

# Use last 100 steps (after transient)
H_map = temporal_entropy_map(history[100:])

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].imshow(history[0], cmap='gray_r')
axes[0].set_title('t = 0 (random)')
axes[1].imshow(history[-1], cmap='gray_r')
```

Comparative Entropy Analysis

Different initial conditions → different entropy trajectories

```
def compare_entropy_trajectories(configs, names, steps=500):
    """Compare H(t) curves for different initial conditions."""
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))

    for grid, name in zip(configs, names):
        hist = evolve(grid.copy(), steps)

        # Density entropy
        H_dens = [density_entropy(h) for h in hist]
        axes[0].plot(H_dens, label=name, linewidth=1.5)

        # Block entropy (k=2)
        H_block = [block_entropy_2d(h, k=2) for h in hist[::10]]
        axes[1].plot(range(0, steps+1, 10), H_block,
                     label=name, linewidth=1.5)

        # Neighborhood entropy (sum)
        H_neigh = [neighborhood_entropy(h) for h in hist[::10]]
        axes[2].plot(range(0, steps+1, 10), H_neigh,
                     label=name, linewidth=1.5)
```

Try: random soups (various ρ_0), Gosper gun, R-pentomino, block field, single glider

```
        ax.set_xlabel('Generation'); ax.set_ylabel('H [bits]')
        ax.set_title(title); ax.legend(fontsize=8)
        ax.grid(alpha=0.3)
```

Entropy Rate & Mutual Information

Advanced measures: temporal correlations and information flow

Entropy Rate

$h = \lim_{n\to\infty} H(X_n \mid X_1, \ldots, X_{n-1})$

How much new information does each generation produce, given the past?

$h = 0$: deterministic (Class I/II)
$h > 0$: unpredictable (Class III/IV)

Mutual Information

$$I(c_{ij}(t) ; c_{ij}(t+\tau)) = H(c_{ij}(t)) + H(c_{ij}(t+\tau)) - H(c_{ij}(t), c_{ij}(t+\tau))$$

Measures temporal predictability.
High I → cell state persists (still lifes).
Low I → cell state uncorrelated (chaos).

```
def mutual_information_temporal(history, tau=1):
    """Average MI between cell state at t and t+tau."""
    T = len(history) - tau
    current = history[:T].reshape(T, -1)      # (T, N)
    future  = history[tau:tau+T].reshape(T, -1)

    # Joint & marginal distributions per cell
    N = current.shape[1]
    MI = np.zeros(N)
    for i in range(N):
        joint = np.zeros((2, 2))
        for a, b in zip(current[:, i], future[:, i]):
            joint[a, b] += 1
```

Putting It All Together

A complete entropy analysis pipeline for GoL

```
def full_entropy_analysis(grid, steps=500, k=2, window=50):
    """Complete entropy analysis of a GoL evolution."""
    history = evolve(grid, steps)

    results = {
        'density_H':      np.array([density_entropy(h) for h in history]),
        'density_rho':    np.array([h.mean() for h in history]),
        'block_H':        np.array([block_entropy_2d(h, k) for h in history]),
        'neighborhood_H': np.array([neighborhood_entropy(h, 'sum')
                                    for h in history]),
        'temporal_map':   temporal_entropy_map(history[-window:]),
        'MI_map':         mutual_information_temporal(history[-window:])[1],
        'population':     np.array([h.sum() for h in history]),
    }
    return history, results

# Run analysis
grid = (np.random.random((100, 100)) < 0.5).astype(int)
history, R = full_entropy_analysis(grid)

# Summary statistics
print(f"Initial density entropy: {R['density_H'][0]:.3f} bits")
print(f"Final density entropy:   {R['density_H'][-1]:.3f} bits")
print(f"Entropy drop:            {R['density_H'][0]-R['density_H'][-1]:.3f} bits")
print(f"Final density:           {R['density_rho'][-1]:.4f}")
print(f"Final block entropy:     {R['block_H'][-1]:.3f} / {k*k} bits")
```

Exercises

1 Entropy vs. Initial Density

30 min

Run GoL from random soups with $\rho_0 \in \{0.1, 0.2, \ldots, 0.9\}$. Plot H_density(t) for each. At what ρ_0 does entropy decrease most? Does the equilibrium density always converge to Flammenkamp's constant?

2 Block Entropy Scaling

45 min

For the settled configuration of a random 200×200 soup, compute H_k for k = 1, 2, 3, 4, 5. Plot H_k / k^2 vs. k. Does it converge? Compare with a purely random grid of the same density.

3 Temporal Entropy Maps

45 min

Evolve a Gosper glider gun for 300 steps. Compute the temporal entropy map. Identify the gun mechanism, glider tracks, and dead zones from the heatmap alone. What is the average temporal entropy of the gun vs. a random soup?

★ Entropy Across Life-like Rules

60+ min

Choose 5 other life-like rules (e.g., HighLife B36/S23, Seeds B2/S, Day & Night B3678/S34678). Compare entropy trajectories from random soups. Which rules show anti-entropic behavior? Can you identify the 'edge of chaos' from entropy alone?

Key Takeaways

1. Density entropy is the simplest measure but misses spatial structure entirely.
2. 2D block entropy (H_k) captures local correlations; larger k → more information about spatial order.
3. Temporal entropy maps reveal the 'activity skeleton' of GoL — still lifes, oscillators, and glider tracks.
4. GoL exhibits anti-entropic behavior: entropy spontaneously decreases from random initial conditions. This is possible because the dynamics are non-reversible (dissipative).
5. Entropy signatures place GoL at the 'edge of chaos' — a special point in rule space that maximizes complexity.

References

Foundational

Shannon, C.E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal.

Wolfram, S. (1984). Computation theory of cellular automata. Comm. Math. Phys. 96(1), 15–57.

Cover, T.M. & Thomas, J.A. (2006). Elements of Information Theory. 2nd ed., Wiley.

Entropy in GoL and 2D CA

Pan, Z. (2010). Emergence from symmetry: A new type of cellular automata. arXiv:1003.3394.

Ruivo et al. (2023). Thermodynamics in Stochastic Conway's GoL. MDPI Dynamics 8(2), 47.

Contreras-Reyes, J.E. (2021). Lerch distribution based on MaxNSEntr: Application to GoL. Chaos, Solitons & Fractals.

Sherrill et al. (2024). Conway's GoL as an Analogue to a Habitable World. arXiv:2410.22389.

Omniperiodicity

Brown, N. et al. (2023). Conway's Game of Life is Omniperiodic. arXiv:2312.02799.

Next: From Local Rules to Global Agents

How the entropy perspective connects cellular automata to multi-agent systems and emergent intelligence

Prof. Mirko Degli Esposti • mirko.degliesposti@unibo.it • University of Bologna